

WB DEVELOPER TECHNICAL CHALLENGE

CUSTOMER SERVICE REQUEST PORTAL

1. OBJECTIVE

Develop a responsive Single-Page Application using React and TypeScript for managing customer service requests.

An OpenAPI 3 specification will be provided describing the Service Request API. Your application must consume this API and use an external OpenID Connect provider, such as Keycloak, Auth0, or another standards-compliant provider, for authentication.

The completed solution must be submitted through a public GitHub repository.

1. REQUIREMENTS

The SPA must allow authenticated users to:

- Sign in and sign out through an OIDC provider.
- View a paginated list of service requests.
- Search requests by title or requester.
- Filter requests by status and priority.
- Sort requests by creation date.
- View the details of a selected request.
- Create a new service request.
- Update the status of an existing request.
- Handle loading, empty, validation, authentication, and API error states.

The interface must be responsive and usable on desktop and mobile devices.

1. TECHNOLOGY

The solution must use:

- React.
- TypeScript.
- Git and GitHub.
- A client-side routing solution.

- An automated testing framework.
- GitHub Actions (CI/CD).

You may choose additional libraries for state management, forms, validation, styling, API access, OIDC integration, and testing. Major choices must be explained in the README.

1. DOCUMENTATION

The repository must include a README.md containing:

- Solution overview.
- Technology and library choices.
- Architecture summary.
- Local setup instructions.
- OIDC provider configuration.
- Environment-variable configuration.
- API-mocking approach.
- Development, lint, test, and build commands.
- Testing strategy.
- GitHub Actions workflow description.
- Security and accessibility considerations.
- Known limitations.

An .env.example file should document the required configuration without containing actual credentials.

1. EVALUATION AREAS

The submission will be assessed on:

- Functional completeness.
- React and TypeScript proficiency.
- Application architecture and maintainability.
- OAS3 contract integration.
- OIDC implementation and security awareness.
- Error handling and resilience.
- Automated testing.
- Responsive design and accessibility.